

Eötvös Loránd Tudományegyetem

Informatikai Kar

Térképtudományi és Geoinformatikai Intézet

Ellenfél alapú támadások (FGSM, PGD) implementációja konvolúciós neurális hálózatokhoz interaktív tesztkörnyezetben

Szerző:
Gabrity Gábor
Programtervező Informatikus BSc

Belső témavezető:
Dr Varga Zsófia
adjunktus

Külső témavezető:
Szabó András Miklós
tanársegéd

Budapest, 2026

Nyilatkozat

Alulírott Gabrity Gábor, az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférését a benyújtástól számított két évig nem kérem.

Budapest, 2026. november 7.

Gabrity Gábor s.k.
hallgató aláírása

Tartalomjegyzék

Rövidítés- és jelölésjegyzék **4**

1. Bevezetés **5**

1.1. Motiváció és a témaválasztás indoklása 5

1.2. A megoldandó feladat rövid, közérthető leírása 5

1.3. A dolgozat célja és hozzájárulásai 5

1.4. A dolgozat felépítése 5

2. Felhasználói dokumentáció **6**

2.1. A megoldott probléma rövid megfogalmazása 6

2.2. A felhasznált módszerek rövid áttekintése 6

2.3. Rendszerkövetelmények 6

2.4. Telepítés és első indítás 6

2.5. A grafikus felhasználói felület bemutatása 6

2.6. Tipikus felhasználási forgatókönyvek 6

2.7. Hibaüzenetek és gyakori problémák 6

3. Fejlesztői dokumentáció **7**

3.1. A probléma részletes specifikációja 7

3.1.1. A modell, a bemenet és a címke 7

3.1.2. A támadás célja 8

3.1.3. Célzott és nem célzott mód 8

3.1.4. A white-box hozzáférési feltétel 8

3.1.5. A szoftver célja és beállítható paraméterei 8

3.1.6. A felhasználói funkciók vázlatos áttekintése 8

3.2. Elméleti háttér és a használt fogalmak definíciója 9

3.2.1. Az osztályozó modell és a logitok 9

3.2.2. Valószínűségek és a modell predikciója 9

3.2.3. Veszteségfüggvény és bemeneti gradiens 10
3.2.4. Perturbáció és normakorlát 10
3.3. Ellenfél alapú támadások áttekintése 11
3.3.1. FGSM (Fast Gradient Sign Method) 11
3.3.2. PGD (Projected Gradient Descent) 12
3.4. Továbbmutató elméleti vonatkozások 13
3.4.1. Linearitás 13
3.4.2. Transzferálhatóság 14
3.4.3. A nyeregpont (saddle point) optimalizálási keret és a lokális maximumok koncentrációja 15
3.4.4. Ellenfél alapú tanítás 16
3.4.5. Madry és társai következtetései 17
3.5. Kapcsolódó munkák és további támadási módszerek 17
3.5.1. Optimalizáció-alapú korai módszerek 17
3.5.2. Univerzális perturbációk 18
3.5.3. Fizikailag megvalósítható támadások 19
3.6. Védekezési stratégiák áttekintése 20
3.6.1. A JPEG-tömörítés mint védekezés 20
3.6.2. A Gauss-elmosás mint védekezés 21
3.7. A rendszer architektúrája 21
3.7.1. Az adatfolyam 22
3.8. A felhasznált technológiák 23
3.9. Tervezési döntések és indoklásuk 24
3.10. Az egyes komponensek megvalósítási részletei 26
3.10.1. `model_manager.py` modul 26
3.10.2. `attack_engine.py` modul 26
3.10.3. `defense_module.py` modul 27
3.10.4. `utils.py` modul 27

4. Tesztelés **29**
4.1. Egységtesztek 29
4.2. Integrációs tesztek 33
4.3. Automatizált GUI tesztek 34
4.4. Funkcionális tesztek és felhasználói validáció 35

4.5. Teszteredmények és értelmezésük 37
4.6. A tesztelés korlátai 37

5. Összefoglalás **38**
5.1. Elért eredmények 38
5.2. Következtetések 38
5.3. Továbbfejlesztési lehetőségek 38

Irodalomjegyzék **39**

Rövidítés- és jelölésjegyzék

1. fejezet

Bevezetés

1.1. Motiváció és a témaválasztás indoklása

1.2. A megoldandó feladat rövid, közérthető leírása

1.3. A dolgozat célja és hozzájárulásai

1.4. A dolgozat felépítése

2. fejezet

Felhasználói dokumentáció

2.1. A megoldott probléma rövid megfogalmazása

2.2. A felhasznált módszerek rövid áttekintése

2.3. Rendszerkövetelmények

2.4. Telepítés és első indítás

2.5. A grafikus felhasználói felület bemutatása

2.6. Tipikus felhasználási forgatókönyvek

2.7. Hibaüzenetek és gyakori problémák

3. fejezet

Fejlesztői dokumentáció

Ebben a fejezetben a megvalósítandó szoftver célját és funkcióit formális, matematikai keretben definiálom, mivel a későbbi fejezetekben tárgyalt algoritmusok (FGSM, PGD) és a védekezési modul is ugyanerre a jelölésrendszerre épülnek majd. Először a probléma szereplőit, a modellt, a bemenetet és a hozzá tartozó címkét vezetem be, majd ezek segítségével pontosan megfogalmazom, mit is jelent az ellenfél alapú perturbáció. Ezután az itt bevezetett jelölésrendszerre épülve sorra bemutatom a két megvalósított támadási algoritmust (FGSM, PGD), a mögöttük álló tervezési döntéseket, a kapcsolódó munkákat, a védekezési stratégiákat, végül a szoftver architektúráját, a felhasznált technológiákat és az egyes komponensek megvalósítását.

3.1. A probléma részletes specifikációja

3.1.1. A modell, a bemenet és a címke

Legyen adott egy f_θ neurális háló alapú osztályozó modell, melynek x egy bemenete és y annak helyes címkéje. Az x bemenet ebben az esetben egy kép pixeleinek valamely három színű kódolása, vektoriális formában, az y pedig a kép helyes osztályának indexe (például $y = 42$ az agámák ImageNet-1k osztályát reprezentálja). Amennyiben a képhez tartozó y helyes címke nem ismert, úgy a modell által meghatározott legvalószínűbb címkét $\hat{y}$-nal jelöljük, és ezt használjuk referenciacímkeként.

3.1.2. A támadás célja

Célunk az x képre egy x_{adv} bemenet előállítása, mely az eredeti x-től szemmel csak alig látható perturbációban különbözik, viszont a modell által adott $\hat{y}_{\text{adv}} = \arg\max_i f_\theta(x_{\text{adv}})_i$ predikció a referenciacímkétől eltér, vagy a klasszifikáció elbizonytalanodik.

3.1.3. Célzott és nem célzott mód

Alapvetően két támadási módot különböztetünk meg, az ún. célzott és nem célzott módot. Nem célzott esetben a támadás célja minél jobban elbizonytalanítani a helyes osztályozást, tehát hogy agáma helyett bármi mást mutasson. Célzott módban azt szeretnénk, hogy egy kijelölt y_{target} osztály irányába változzon a képünk (pl. a komodói varánusz osztálya felé).

3.1.4. A white-box hozzáférési feltétel

A támadás feltétele, hogy az f_θ modell gradienseihez és paramétereihez teljes hozzáférésünk legyen. Ezt white-box hozzáférésnek nevezzük az AI red teaming kontextusában.

3.1.5. A szoftver célja és beállítható paraméterei

E szoftver célja a fenti ellenfél alapú támadó tesztkörnyezet szoftveres megvalósítása, különböző támadási algoritmusokkal (FGSM, PGD). A szoftver a PyTorch (torchvision) előtanított modelljeire épül, mely architektúra tetszőleges torchvision-modellel bővíthető. Jelenleg kettő van integrálva (ResNet50, MobileNetV2). Beállíthatóak a futás paraméterei:

- modell: ResNet50 vagy MobileNetV2, amelyen a támadást végrehajtjuk,
- ϵ, mely meghatározza a perturbáció maximális mértékét, tehát azt, hogy egy pixelen legfeljebb mekkora változást tapasztalhatunk, azaz a változás felső határát,
- célosztály, y_{target}, ha a mód célzott,
- iterációszám (PGD esetén hány lépésig engedjük futtatni az algoritmust).

3.1.6. A felhasználói funkciók vázlatos áttekintése

A fenti specifikációt a szoftver az alábbi, a felhasználó számára közvetlenül elérhető funkciókkal valósítja meg:

- a fájlrendszerről kiválasztott képet legördülő menüből kiválasztott modellel klasszifikáljuk,
- támadási algoritmus és paraméterek meghatározásával perturbációt és ellenfél alapú képet generálunk,
- megjelenítjük az eredeti képet, a perturbációt és az ellenfél alapú képet, továbbá mindkét (eredeti és ellenfél alapú) képet osztályozzuk is,
- a védekezési modulban JPEG-tömörítést vagy más képfeldolgozási szűrőt alkalmazhatunk az ellenfél alapú perturbáció hatásának utólagos csökkentésére vagy semlegesítésére,
- az eredeti és az ellenfél alapú kép PNG formátumban letölthető, a mentett képek pedig visszatölthetőek.

3.2. Elméleti háttér és a használt fogalmak definíciója

3.2.1. Az osztályozó modell és a logitok

Jelöljük $f_\theta(x) = z$-vel a modell outputjaként előállt valós számok tömbjét, ahol $z \in \mathbb{R}^C$ és C az osztályok száma (ImageNet-1k esetén $C = 1000$). Ezt logitvektornak is nevezzük, mely elnevezés a logisztikus regresszióból származik, ahol a $\text{logit}(p) = \ln \frac{p}{1-p}$ függvény egy $p \in (0, 1)$ valószínűséget képez le a valós számegyenesre. A z_i komponensek páronkénti különbsége éppen az i-edik és j-edik osztály esélyhányadosának logaritmusával egyezik meg, $z_i - z_j = \ln \frac{p_i}{p_j}$, innen származik az elnevezés.

3.2.2. Valószínűségek és a modell predikciója

Mivel a z logitok önmagukban nem valószínűségek, a softmax függvénnyel alakítjuk át őket valószínűségi eloszlássá:

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}, \qquad i = 1, \ldots, C.$$

Tehát p_i a modell szerint annak a valószínűsége, hogy x az i-edik osztályba tartozik. A modell által meghatározott legvalószínűbb osztály indexét a legnagyobb valószínűségű komponens alapján kapjuk:

$$\hat{y} = \arg\max_i p_i.$$

3.2.3. Veszteségfüggvény és bemeneti gradiens

A $J(\theta, x, y)$ veszteségfüggvény a modell predikciója és a megadott helyes címke közötti eltérést méri. Esetünkben ez a keresztentrópia-veszteség

$$J(\theta, x, y) = -\log \frac{e^{z_y}}{\sum_{i=1}^{C} e^{z_i}},$$

ahol $z = f_\theta(x)$ a logitvektor és y a helyes osztály indexe. A veszteségfüggvény súlyok szerinti gradiense, $\nabla_\theta J(\theta, x, y)$, az a gradiensvektor, amelyet a hiba-visszaterjesztés (angolul *backpropagation*) során a tanítás közben használunk a súlyok módosítására. Itt a gradiensszámítást egy további lépéssel a bemeneti rétegig visszük – azaz a hiba-visszaterjesztés folyamatát a hálón teljesen végigvezetjük –, és megkapjuk a bemeneti vektorra vonatkozó érzékenységet, $\nabla_x J(\theta, x, y)$. Ez a mennyiség azt mutatja meg, hogy egy adott bemeneti pixel vagy jellemző változása milyen mértékben befolyásolja a veszteséget, ezért ennek a mennyiségnek központi szerepe lesz a későbbiekben bemutatott, ellenfél alapú példát generáló algoritmusok leírásában.

3.2.4. Perturbáció és normakorlát

Az ellenfél alapú képek definíciójánál szokás a perturbáció nagyságát valamely ℓ_p normával jellemezni; ebben a dolgozatban a ℓ_∞ normát használjuk. Egy perturbált kép általában az

$$x_{\text{adv}} = x + \delta$$

formában írható le, ahol δ a zaj, vagyis a képpontokban okozott változás. A ℓ_∞ normát a

$$\|v\|_\infty = \max_i |v_i|$$

kifejezés definiálja, azaz a legnagyobb abszolút értékű komponens adja a norma értékét. A perturbáció nagyságát ezzel a normával korlátozzuk:

$$\|\delta\|_\infty \leq \epsilon.$$

Ez esetünkben nagyon kézenfekvő, mivel a bemeneti kép minden pixelén (minden δ_i komponensen) külön-külön korlátozza a megengedett változást.

3.3. Ellenfél alapú támadások áttekintése

A mély neurális hálók, köztük a képfelismerő modellek is, különösen érzékenyek a bemeneti adatok kicsi, ám célzott módosításaira [1]. Az ilyen módosításokat ellenfél alapú perturbációknak nevezzük. Ezeknek a támadásoknak két algoritmusát fogjuk itt átvenni, melyek a taxonómiában a korábban említett white-box támadások közé tartoznak. A következő fejezetekben egy egylépéses ellenfél alapú támadást mutatunk be (FGSM), utána erre építve egy többlépéses algoritmust (PGD).

3.3.1. FGSM (Fast Gradient Sign Method)

Az FGSM algoritmust Goodfellow és szerzőtársai a Google-nél végzett kutatásaik alapján javasolták [1]. Az előző fejezetben megjegyeztem, hogy a $\nabla_x J(\theta, x, y)$ veszteségfüggvény gradiense az input értékre való érzékenységet jelenti. Természetes ötlet, hogy a bemenetet a gradienssel ellentétes irányba mozdítsuk el, ami várhatóan növeli a veszteségfüggvény értékét, azaz rontja a helyes osztályra adott predikciót. Ehhez feltesszük, hogy a modell az x pont kellően kis környezetében közelítőleg lineárisan viselkedik – ez Goodfellow és társai linearitási elmélete, amely szerint a modern neurális hálók, részben a szakaszonként lineáris aktivációk (pl. ReLU) miatt, lokálisan jól közelíthetők egy lineáris függvénnyel, még ha globálisan erősen nemlineárisak is [1]. Az FGSM algoritmusban az elmozdítást egyszerűen csak az előjel-függvény (sign) ϵ-szorosával tesszük meg, tehát nem vesszük figyelembe a gradiens nagysága által adott érzékenységet, csak az irányát. Ily módon az $\epsilon \cdot \text{sign}(\cdot)$ minden egyes x_i komponensre kihasználja a teljes ϵ költségvetésünket. Tehát az FGSM ellenfél alapú példát a következő módon kapjuk meg:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}\Big(\nabla_x J(\theta, x, y)\Big).$$

Célzott változat

Az alapváltozat a megadott helyes osztály veszteségét igyekszik maximalizálni, tehát minél inkább mást mutasson, mint ami a képen van. A célzott változatnál egy y_{target} célosztályt adunk meg, és azt szeretnénk, hogy a modell ebbe az osztályba sorolja az ellenfél alapú képet.

$$x_{\text{adv}} = x - \epsilon \cdot \text{sign}\Big(\nabla_x J(\theta, x, y_{\text{target}})\Big).$$

Tehát a képlet csak annyiban változik, hogy távolodás helyett az y_{target}-hoz tartozó veszteség minimuma felé közeledni szeretnénk, és y helyett y_{target}-ot használjuk.

Előnyök és korlátok

Mivel a gradienst a tanításban is használt hiba-visszaterjesztés segítségével hatékonyan tudjuk kiszámolni, ezért a támadás gyors. Hátránya, hogy csak egyetlen, durva lépést tesz a megfelelő irányba, ezért – főleg az iteratív módszerekhez, például a PGD-hez képest – kevésbé hatékonyan képes megtéveszteni az osztályozót.

3.3.2. PGD (Projected Gradient Descent)

Ezt a módszert Madry és társai vezették be [2], és lényegében az FGSM egylépéses ötletének iteratív általánosítása. Nem egyetlen nagy lépést teszünk a gradiens előjele irányába, hanem ugyanabba az irányba több, kisebb lépést hajtunk végre egymás után. A motiváció emögött az, hogy az FGSM linearitási feltevése csak x kellően kis környezetében állja meg a helyét, ezért előnyösebb, ha T darab kisebb, α nagyságú lépéssel dolgozunk, és a gradienst minden lépésben újraszámoljuk. Legyen $S = \{\delta : \|\delta\|_\infty \leq \epsilon\}$ a megengedett perturbációk halmaza; ekkor $x + S$ adja meg az x körüli, ϵ sugarú gömbbe eső, engedélyezett bemenetek halmazát. (ℓ_∞ norma esetén ez geometriailag valójában egy kocka, de általános ℓ_p értelemben vett gömbként hivatkozunk rá.) Mivel az iteráció túlléphet $x + S$ korlátján, az algoritmus minden lépés után egy projekcióval visszavetíti az eredményt erre a halmazra. Ez FGSM esetén a sign függvény miatt automatikusan teljesült, itt viszont T és α megválasztásától függően külön biztosítani kell. Az algoritmus formális definíciója tehát

$$x^{t+1} = \Pi_{x+S}\Big(x^t + \alpha \cdot \operatorname{sign}(\nabla_x J(\theta, x^t, y))\Big), \qquad x^0 = x,$$

ahol Π_{x+S} az $x + S$ halmazra való projekció.

Célzott változat

Az FGSM-hez hasonlóan itt is definiálunk egy célzott változatot, amely az ottanival analóg módon adható meg:

$$x^{t+1} = \Pi_{x+S}\Big(x^t - \alpha \cdot \operatorname{sign}(\nabla_x J(\theta, x^t, y_{\text{target}}))\Big).$$

Véletlen kiindulási pont

Madry és társai kísérletileg mutatták meg, hogy a veszteségfelszín lokális maximumai jól koncentráltak, így egy véletlen kezdőpontból indított PGD is megbízhatóan jó megoldást talál [2] (a jelenség hátterét a 3.4.3. alfejezet tárgyalja részletesen). Ennek a modellek

robusztusságának tesztelésében is van szerepe: mivel a veszteségfelszínnek több, egymástól távol eső lokális maximuma is lehet, egy fix (pl. magából x-ből induló) kezdőpont esetén a szerencsének nagy szerepe lehetne, és az eredmény nem feltétlenül mutatná a modell valódi sebezhetőségét. Ezért a véletlen kezdőpontot az ϵ sugarú gömbben egyenletes eloszlással választjuk meg:

$$x^0 = \Pi_{x+S}(x + \eta), \qquad \eta \sim \mathcal{U}(-\epsilon, \epsilon)^d.$$

Az α és T paraméterek megválasztása

A PGD viselkedését az ϵ perturbációs korláton felül két további paraméter határozza meg. T az iterációk számát jelöli, ez adja meg az algoritmus számítási költségét, ugyanakkor több iteráció jellemzően erősebb támadást eredményez. Az α lépésnagyságot úgy kell megválasztani, hogy túl kicsi érték esetén lassú a konvergencia, túl nagy érték esetén pedig instabilitáshoz vezethet. Tipikus választás: $\alpha = \epsilon/T$ vagy $\alpha = 2{,}5 \cdot \epsilon/T$ [2].

Előnyök és korlátok

A PGD az FGSM-nél lényegesen erősebb támadásokat képes generálni, mivel több kis lépésben közelíti a lokális optimumot. A véletlen kezdőponttal kombinálva különösen hatékony a robusztusság kiértékelésében. Hátránya, hogy T-szer annyi gradiens-számítást igényel, mint az FGSM, így nagyobb a számítási költsége.

3.4. Továbbmutató elméleti vonatkozások

Ebben a fejezetben azokat az elméleti vonatkozásokat foglalom össze, amelyek nem képezik szigorúan a dolgozatban megvalósított szoftver részét, ám fontos kiegészítői annak.

3.4.1. Linearitás

Goodfellow és társai az ellenfél alapú példák létezésére nem a neurális hálók feltételezett, extrém nemlinearitásán vagy túltanulásán alapuló magyarázatot adnak. Ehelyett ennek éppen az ellenkezőjét állítják. Szerintük a jelenség hátterében a modern modellek meglepően lineáris viselkedése áll [1].

A legegyszerűbb esetben tekintsünk egy lineáris klasszifikátort w súlyvektorral, tehát egy egyrétegű modellt. Vizsgáljuk az ellenfél alapú $\tilde{x} = x + \eta$ bemenet aktivációját.

$$w^\top \tilde{x} = w^\top x + w^\top \eta.$$

A $\|\eta\|_\infty \leq \epsilon$ korlát mellett a $w^\top \eta$ növekmény akkor maximális, ha $\eta = \epsilon \cdot \text{sign}(w)$. Ez pontosan az FGSM képletének lineáris modellre vonatkozó speciális esete. Ha w dimenziója n, és egy komponensének átlagos nagysága m, akkor az aktiváció növekménye ϵmn nagyságrendű. Mivel $\|\eta\|_\infty$ maga nem nő a dimenziószámmal, az általa okozott aktivációváltozás viszont lineárisan nő n-nel, ezért elég nagy dimenziójú bemenet esetén sok, egyenként elhanyagolhatóan kicsi (a pixelek 8 bites kódolási pontosságánál is kisebb) elmozdítás összegződve tetszőlegesen nagy kimeneti változást okozhat. Ez azt mutatja, hogy már egy egyszerű lineáris modellnek is lehetnek ellenfél alapú példái, ha a bemenete elég nagy dimenziójú.

Goodfellow és társai szerint a modern neurális hálók (pl. ReLU, maxout, LSTM) tudatosan lineáris viselkedésre vannak tervezve, mivel ez megkönnyíti az optimalizálásukat. Emiatt ezek a hálók, megfogalmazásuk szerint "túl lineárisak" ahhoz, hogy ellenálljanak egy lineáris ellenfél alapú perturbációnak. Ez a megfigyelés adja a korábban bemutatott FGSM elméleti alapját.

3.4.2. Transzferálhatóság

Az ellenfél alapú perturbációk egyik legfontosabb tulajdonsága a transzferálhatóság, ami azt jelenti, hogy egy adott f_θ modellre kiszámított x_{adv} sokszor egy másik, eltérő architektúrájú, más inicializálású, vagy akár más adathalmazon tanított modellt is megtéveszthet. Tehát a megtévesztés úgy is megtörténhet, hogy a perturbációt nem az ő gradienseiből számoltuk, azaz lehetővé teszi a black-box támadásokat. Ehhez elég egy hasonló feladatra tanított modellen white-box módon (pl. FGSM vagy PGD segítségével) előállítani a perturbációt, majd azt a célmodellen felhasználni. Goodfellow és társai a jelenség magyarázatát is a 3.4.1. alfejezetben bemutatott linearitási hipotézisre vezetik vissza. Szerintük a különböző architektúrájú modellek ugyanazon adathalmazon hasonló, közelítőleg lineáris döntési határt tanulnak meg, ezért egy adott modellre számolt perturbáció a hasonló döntési határ miatt más modelleket is megtéveszt [1].

Goodfellow és társai kísérletileg igazolták a transzferálhatóságot. Az FGSM-mel az eredeti modellre generált ellenfél alapú példák 19,6%-os hibaarányt okoztak egy másik, ellenfél alapú tanítással robusztusabbá tett modellen, míg az utóbbi modellre generált példák 40,9%-os hibaarányt okoztak az eredetin. Ezek szerint a robusztusabb modellről származó perturbációk erősebbnek bizonyultak a transzfer irányban is [1].

Madry és társai ezt a jelenséget rendszerezettebben, több modellpár és mindkét vizs-

gált algoritmus (FGSM, PGD) összehasonlításával vizsgálták, és két fő tendenciát azonosítottak. Minél erősebb az a támadó (adversary), amellyel a forrásmodellt tanították, illetve minél nagyobb a modell kapacitása, annál kevésbé transzferálhatóak a rá generált ellenfél alapú példák más modellekre [2]. Ennek oka, hogy a nagyobb kapacitású, erősebb támadó ellen (pl. PGD-vel) tanított modellek veszteségfelszíne kevésbé korrelál a többi modellével, így a rájuk kiszámított gradiensirány egyre kevésbé lesz általánosan érvényes.

3.4.3. A nyeregpont (saddle point) optimalizálási keret és a lokális maximumok koncentrációja

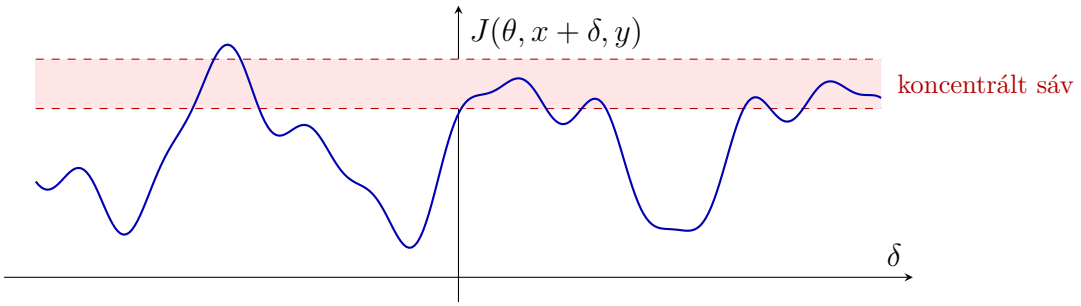
Madry és társai az ellenfél alapú robusztusságot a robusztus optimalizálás nyelvén, egy úgynevezett nyeregpont- (angolul *saddle point*, más néven min-max) feladatként fogalmazzák meg [2]:

$$\min_{\theta} \ \rho(\theta), \qquad \rho(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\Big[\max_{\delta\in S} J(\theta, x+\delta, y)\Big],$$

ahol $S = \{\delta : \|\delta\|_\infty \leq \epsilon\}$ a korábban bevezetett, megengedett perturbációk halmaza. A belső maximalizálási feladat a támadás problémáját írja le. Ez az adott θ paraméterű modellhez a veszteséget maximalizáló δ perturbáció megtalálását jelenti. A külső minimalizálási feladat a védekezés problémája, tehát olyan θ paraméterek keresése, amelyek mellett a belső maximalizálás által adott a legrosszabb esetű veszteség is minél kisebb. Ez a maximum minimalizálás pontosan az ellenfél alapú tanítás célja.

Mivel a belső feladat egy erősen nem konkáv függvény maximalizálása, elsőre nem várnánk, hogy megbízhatóan megoldható. Az ilyen feladatoknak sokszor egymástól távol eső lokális maximumai lehetnek, és nincs rá garancia, hogy egy gradiens alapú módszer a globális maximum közelébe jut. Madry és társai ezt a kérdést kísérletileg vizsgálták. Ebben sok, független, véletlenszerűen választott kezdőpontból indítottak PGD-t ugyanazon x bemenet ϵ sugarú gömbön belül, és a talált lokális maximumok tulajdonságait figyelték [2]. Azt találták, hogy bár a talált lokális maximumok ténylegesen különbözőek, az általuk elért veszteségértékek meglepően szűk sávban koncentrálódnak, extrém kiugró érték nélkül. A veszteség emellett viszonylag kevés iteráció után platóra ér, tehát a PGD gyorsan konvergál egy, a globálishoz közeli értékhez, függetlenül a konkrét kezdőponttól. Ez a jelenség rokon azzal a tapasztalati megfigyeléssel, hogy a neurális hálók tanítása azért működik jól annak ellenére, hogy a veszteségfelszín is sok lokális minimumot tartalmaz. Ezek a lokális minimumok ugyanis szintén hasonló értékűek egymáshoz. Ugyanez a

regularitás jelentkezik itt a bemeneti perturbáció szerinti belső maximalizálásra vonatkozóan is. Ez azt mutatja, hogy a belső feladat a nem konkávsága ellenére is jól kezelhető, „szelíd” szerkezetű (3.1. ábra).



3.1. ábra. Sematikus ábra a veszteségfüggvény δ perturbáció szerinti. Sok, egymástól távol eső lokális maximum létezik, ezek értékei egy szűk sávban koncentrálódnak.

Ez a felismerés adja a PGD elméleti indoklását mint a legrosszabb esetű, kizárólag elsőrendű (gradiens-) információt felhasználó ellenfelek közül a legerősebbet, az ún. „univerzális első rendű ellenfelet”. Ez azért van, mert a kezdőponttól függetlenül megbízhatóan hasonlóan jó, a globálishoz közeli megoldást talál.

3.4.4. Ellenfél alapú tanítás

A 3.4.3. alfejezetben bemutatott nyeregpont-feladat külső minimalizálási része pontosan az ellenfél alapú tanítás elméleti megfogalmazása. Ez azon θ paraméterek keresését jelenti, amelyek mellett a belső maximalizálás által adott, legrosszabb esetű veszteség is minél kisebb. Ennek gyakorlati megvalósításához a tiszta x kép mellett a hozzá tartozó x_{adv} ellenfél alapú képet is bevonjuk a veszteségfüggvénybe, és a modell paramétereit az így kapott kombinált célfüggvényre optimalizáljuk.

Goodfellow és társai az FGSM-mel előállított x_{adv} perturbációra a következő kombinált veszteségfüggvényt javasolták [1]:

$$\tilde{J}(\theta, x, y) = \lambda\, J(\theta, x, y) + (1-\lambda)\, J(\theta, x_{\text{adv}}, y), \qquad \lambda = 0{,}5.$$

Az új veszteségfüggvény tehát a tiszta és az ellenfél alapú minta hibáját egyaránt bünteti, méghozzá úgy, hogy x_{adv}-ot minden tanítási lépésben a modell aktuális θ paramétereivel újraszámolják. Így tehát a modell a tanítás során folyamatosan fejlődik, és az aktuális állapota által generált ellenfél alapú példákon tanul tovább dinamikusan, nem pedig egy előre legyártott, statikus ellenfél alapú halmazon. Kísérleteik megmutatták, hogy ez

egy hatékony módszer. Az FGSM-mel előállított ellenfél alapú példákon mért hibaarány 89,4%-ról 17,9%-ra csökkent.

Madry és társai megmutatták, hogy PGD-vel is alkalmazható az ellenfél alapú tanítás, és erősebb védekezést eredményez [2]. Itt azonban a tiszta és ellenfél alapú veszteséget kombináló FGSM-formulával ellentétben nincs szó keverésről. A tanítás során a tiszta x kép helyett kizárólag a hozzá tartozó, PGD-vel előállított x_{adv}-on számolják a veszteséget, tehát a modell csak az ellenfél alapú bemeneteken tanul. Mivel a PGD erősebb támadás, mint az FGSM, az ellene edzett modell a gyengébb, egylépéses támadásokkal szemben is ellenállóbb marad, míg a fordítottja nem feltétlenül igaz.

3.4.5. Madry és társai következtetései

Madry és társai eredményei azt igazolják, hogy a mély neurális hálók ellenálló képessége az ellenfél alapú támadásokkal szemben megbízhatóan növelhető. Kísérleti eredményeik alapján megbízható ellenfél alapú tanítási módszerek tervezhetők [2]. Ennek egyik kulcsfontosságú alapja a fenti felismerés, miszerint a PGD adja az univerzális első rendű ellenfelet. Mindezek alapján a szerzők úgy vélik, hogy az ellenfél alapú támadásokkal szemben robusztus mély tanulási modellek a jelenlegi eszközökkel is elérhetőek. Kísérleti eredményeik ugyanakkor azt is megmutatták, hogy ez a robusztusság adathalmaz-függő. MNIST adathalmazon (szürkeárnyalatos kézzel írott számjegyek) a hálóik igen robusztusnak bizonyultak, és nagy pontosságukat erős ℓ_∞-korlátos ellenfelek és nagy perturbációk esetén is megtartották. A bonyolultabb CIFAR10 adathalmazon (színes, természetes fényképek 10 kategóiában) a kísérleteik ezt a szintet még nem érték el. Az eredményeik azonban itt is jelentős javulást mutattak a robusztusságban.

3.5. Kapcsolódó munkák és további támadási módszerek

3.5.1. Optimalizáció-alapú korai módszerek

Az ellenfél alapú perturbáció fogalmának első dokumentált megjelenése Szegedy és társai nevéhez fűződik [3]. Módszerük a támadást egy explicit optimalizálási feladatként fogalmazza meg. A cél egy olyan, ℓ_2 normában minimális δ perturbáció megtalálása, amely

mellett a modell predikciója egy előre megadott y_{target} célosztályra változik.

$$\min_{\delta} \|\delta\|_2 \qquad \text{úgy, hogy} \qquad \arg\max_i f_\theta(x+\delta)_i = y_{\text{target}}, \quad x+\delta \in [0,1]^n.$$

Ezt a feltételes feladatot Szegedy és társai a korlátozott memóriaigényű Broyden–Fletcher–Goldfarb–Shanno módszer (L-BFGS) egy dobozkorlátos változatával közelítették, amely egy kvázi-Newton típusú numerikus optimalizálási technika. Kísérleteikben mutatták meg először a korábban tárgyalt transzferálhatóságot, mint a neurális hálók egyik tulajdonságát.

Ez a korai módszer white-box hozzáférést és gradiens alapú optimalizálást igényelt, hasonlóan a korábban bemutatott FGSM-hez és PGD-hez. A lényegi különbség az, hogy itt egy konvergenciáig futó optimalizálási feladatot kell megoldani. Ez számításigényesebb, mint az FGSM egyetlen lépése vagy a PGD rögzített iterációszáma. Ez a számítási költség adta a motivációt Goodfellow és társai számára egy hatékonyabb, egylépéses módszer kidolgozásához, amit a korábban bemutatott FGSM-ként ismerünk [1]. A NIST 2025-ös, ellenfél alapú gépi tanulásról szóló átfogó jelentése szerint jól dokumentált történeti fejlődési láncot alkot a szakirodalomban az optimalizáció-alapú korai módszerektől (Szegedy és társai) az FGSM-en és Kurakin és társai iteratív FGSM-változatán át egészen a PGD-ig [4].

3.5.2. Univerzális perturbációk

Az eddigi módszerek mindegyike digitális támadás, a δ perturbációt egyetlen konkrét x bemenetre számolja ki, tehát minden új képhez elvégzi a gradiensszámítást. Moosavi-Dezfooli és társai ezzel szemben megmutatták, hogy képek egy adott halmazára létezik egyetlen, kép-független (*image-agnostic*) perturbációs vektor is, amely a képek jelentős részére alkalmazva önmagában téves osztályozást eredményez [5]. Az ilyen perturbációt univerzálisnak nevezik. Formálisan, ha μ a bemeneti képek eloszlása és $\hat{y}(x)$ a modell által adott predikció, akkor egy v perturbációs vektort keresnek, amelyre

$$\|v\|_p \le \xi \qquad \text{és} \qquad \Pr_{x\sim\mu}\left[\hat{y}(x+v) \neq \hat{y}(x)\right] \ge 1-\delta,$$

ahol ξ a megengedett perturbáció nagyságát korlátozza, δ pedig a kívánt tévesztési arányt (*fooling rate*) határozza meg.

A szerzők egy iteratív algoritmust javasolnak a v vektor előállítására, amely a tanítóképek egy X részhalmazán dolgozik, és képenként addig módosítja a már felhalmozott

perturbációt, amíg az adott kép át nem kerül a döntési határ túloldalára, minden lépés után visszavetítve az eredményt a ξ sugarú ℓ_p gömbre. Az eljárás lényegi újdonsága, hogy a v vektort elég egyszer kiszámítani egy viszonylag kis képhalmazon, ezt követően új bemenetekre való alkalmazásához elegendő a vektor egyszerű hozzáadása, gradienssszámítás vagy újabb optimalizálás nélkül.

Kísérleteik szerint egy, néhány ezer (akár csak néhány száz) képen számolt univerzális perturbáció a validációs halmaz eddig nem látott képeinek túlnyomó részén, esetenként 80–90 százalék felett is, téves osztályozást eredményez, annak ellenére, hogy szemmel alig észrevehető.

3.5.3. Fizikailag megvalósítható támadások

Az eddig tárgyalt módszerek digitális támadások, a perturbációt közvetlenül a modellnek átadott kép pixeleire számolják ki. A fizikailag megvalósítható támadások ezzel szemben olyan perturbációkat állítanak elő, amelyek a valós környezetben is kivitelezhetőek.

Sharif és társai egy nyomtatott szemüvegkeret mintázatát optimalizálták úgy, hogy azt viselve a támadó elkerülje a felismerést, vagy más személynek adja ki magát [6]. A perturbációt egy gradiens alapú optimalizálással, egy simasági (*total variation*) megszorítással kiegészítve állították elő, amely biztosítja, hogy a mintázat nyomtatható és kamerával visszafényképezhető maradjon. Ezt a nyomtatott szemüveget felvéve a résztvevők az esetek nagy részében sikeresen elkerülték a felismerést, egyes célszemélyeknél a megszemélyesítés is jelentős arányban sikerült.

Eykholt és társai egy hasonló elven működő, közlekedési táblák elleni támadást dolgoztak ki, Robust Physical Perturbations (RP2) néven [7]. A perturbációt nem egy kép pixeleire, hanem a fizikai környezet változatosságát (eltolás, elforgatás, megvilágítás, távolság) modellező eloszlásra optimalizálják, így az a különböző körülmények között készült felvételeken is robusztus marad. Fekete-fehér matricák formájában egy valódi Stop táblára helyezett ilyen formájú perturbáció tábla típusát tévesen azonosította be. Laboratóriumi teszteken teljes, a mozgó járműből rögzített videón pedig igen magas sikerességi arányt értek ezzel el.

Chen és társai a fizikailag megvalósítható támadások körét objektumdetektorokra terjesztették ki, ShapeShifter néven [8]. Ez nehezebb feladat, mert a detektornak egyszerre több, különböző méretű régióban (*bounding box*) kell tévesen felismernie az objektu-

mot. A szerzők az úgynevezett várható érték a transzformáció felett (*Expectation over Transformation*) technikát adaptálták erre a feladatra, és az így létrehozott, nyomtatott Stop táblák a Faster R-CNN detektort a valóságban, különböző távolságokból és szögekből felvéve is következetesen megtévesztették.

A közös tanulság, hogy itt a robusztusságot nem elegendő egyetlen rögzített bemenetre biztosítani, a perturbációnak a fizikai körülmények (nézőpont, távolság, megvilágítás, kamera- és nyomtatási torzítás) teljes változatosságával szemben is meg kell maradnia, ami a támadási felületet a valós, fizikai környezetbe is kiterjeszti, és az önvezető járművek vagy beléptető rendszerek biztonságát közvetlenül érinti.

3.6. Védekezési stratégiák áttekintése

Most pár egyszerűbb védekezési stratégiát nézünk, amelyek a bemeneti képet alakítják át pont a klasszifikáció előtt, remélve, hogy az átalakítás eltávolítja, de legalább csökkenti a perturbáció hatását. A szoftver védekezési moduljában két ilyen módszert, a JPEG-tömörítést és a Gauss-elmosást nézzük itt át. Közös bennük, hogy egyszerűek, nem igényelnek újratanítást vagy hozzáférést a modell gradienseihez, viszont csak kisebb perturbációjú támadások ellen biztosítanak védelmet. Céljuk elsősorban szemléltető, illetve a perturbáció utólagos csökkentésének demonstrálására szolgáló eszközként tekintendő, nem pedig egy megbízható, önmagában elégséges védelmi mechanizmusként.

3.6.1. A JPEG-tömörítés mint védekezés

Dziugaite és társai kifejezetten a JPEG-tömörítés ellenfél alapú perturbációkra gyakorolt hatását vizsgálták [9]. Hipotézisük szerint a természetes képek egy alacsony dimenziós altérben, az adataltérben helyezkednek el, egy ellenfél alapú perturbáció viszont jellemzően kimozdítja a képet ebből az altérből. Mivel a képosztályozási adathalmazok szinte kivétel nélkül JPEG formátumú képekből állnak, egy újratömörítés a perturbált képet visszavetítheti egy, az adataltérnél tágabb, de azt magába foglaló altérbe, az úgynevezett JPEG-altérbe. Feltételezésük szerint ez az újratömörítés legalábbis részben visszaállíthatja a klasszifikáció helyességét.

Eredményeik szerint kis magnitúdójú perturbáció ($\epsilon = 1/255$) esetén a JPEG-tömörítés jelentős mértékben, bár nem teljesen, visszaállította a klasszifikációs pontosságot. Nagyobb magnitúdójú perturbációk ($\epsilon = 5/255$ és $\epsilon = 10/255$) esetén az újratömö-

rítés hatása elenyésző volt, a pontosság nem tért vissza a tiszta képekre jellemző szintre. Ebből azt a következtetést vonták le, hogy a JPEG-tömörítés önmagában nem nyújt megbízható védelmet az ellenfél alapú perturbációkkal szemben, hatékonysága erősen függ a perturbáció nagyságától.

3.6.2. A Gauss-elmosás mint védekezés

A Gauss-elmosás egy szűrő, aminél minden pixel értéket a szomszédos pixelek átlagaként állítjuk elő, azaz letompítjuk azt. Dziugaite és társai megemlítik, hogy Kurakin és társai egy korábbi munkájukban a JPEG-tömörítés mellett vizsgálták a Gauss-elmosást, valamint a fényerő és a kontraszt módosítását is, vagy akár a fizikai világban (nyomtatás és újrafényképezés) átalakított módszereket, mint lehetséges védekezési módszereket [9]. Ez a hivatkozott vizsgálat azt találta, hogy az FGSM-perturbációk viszonylag ellenállóak az ilyen képtranszformációkkal szemben.

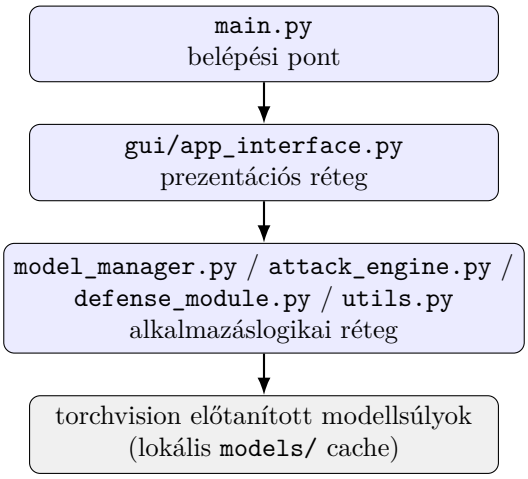
3.7. A rendszer architektúrája

A szoftver egyetlen, önálló asztali alkalmazásként fut, hálózati backend nélkül, ezért architektúrája nem a megszokott kliens–szerver felosztást követi, hanem a felelősségek szerint különül el belső rétegekre. A rétegek a következők, a belépési ponttól a legmélyebb logikáig haladva:

- **Belépési pont** (`main.py`): egyetlen feladata az alkalmazás ablakának példányosítása és a Tkinter eseményhurok elindítása.
- **Prezentációs réteg** (`gui/app_interface.py`): CustomTkinter alapú grafikus felület, amely a felhasználói bemenetet (gombok, csúszkák, célosztály-kereső) eseménykezelőkre köti. Ez kezeli az alkalmazás állapotát (betöltött kép, utolsó támadás eredménye, perturbáció), és a hosszabb futású támadást külön szálra szervezi ki, hogy a felület a számítás alatt is válaszképes maradjon.
- **Alkalmazáslogikai réteg** (`model_manager.py`, `attack_engine.py`, `defense_module.py`, `utils.py`): ez a réteg tartalmazza a prezentációs réteg által hívott legalsó szintű műveleteket, a modellbetöltést, -cache-elést, a támadási algoritmusokat, a védekezési szűrőket és a kép-előfeldolgozást.

- **torchvision modellek**: ez a réteg már nem képezi az alkalmazás szigorúan vett részét. A különböző modelleket egy külső torchvision könyvtár segítségével érjük el.

A rétegek közötti függőség egyirányú, a prezentációs réteg ismeri és hívja az alkalmazáslogikai réteg függvényeit, fordítva viszont az alkalmazáslogikai réteg semmit sem tud a GUI létezéséről. Ez a függetlenség lehetővé teszi, hogy az üzleti logika komponensei önállóan, a felület felépítése nélkül is egységtesztelhetők legyenek. A 3.2. ábra ezt a rétegződést és a fő adatfolyást szemlélteti.



3.2. ábra. A szoftver rétegei és a köztük lévő függés iránya.

3.7.1. Az adatfolyam

Egy tipikus támadás végrehajtása során az adat a következő lépéseken halad végig:

1. A GUI a kiválasztott modellnév alapján lekéri a megfelelő `ModelBundle`-t a `ModelManager`-től, és ebből összeállítja az `AttackContext`-et, amely a modellt és a normalizációs paramétereket (átlag, szórás) fogja össze.

2. Az „Attack” gomb megnyomásakor a GUI egy külön szálon meghívja az `attack_engine.attack()` diszpécser függvényt a betöltött kép tenzorával, a kiválasztott módszerrel (FGSM vagy PGD) és paraméterekkel.

3. Az `attack_engine` a kapott kontextus segítségével végrehajtja a támadást, majd egy `AttackResult`-ot (ellenfél alapú kép, perturbáció, tényleges iterációszám) ad vissza.

4. A háttérszál a főszálra ütemezve frissíti a felületet, megjeleníti az ellenfél alapú képet, a felnagyított perturbációt, valamint az újraszámolt predikciókat.

5. Amennyiben a felhasználó ezt követően védekezést is alkalmaz, a `defense_module` az ellenfél alapú képre Gauss-elmosást vagy JPEG-tömörítést hajt végre, és az eredmény ugyanazon az úton kerül vissza a felületre.

Az alkalmazásnak nincs saját adatbázisa vagy perzisztens állapota. Az egyetlen, lemezen megmaradó adat az előtanított modellek, amelyet a `ModelManager` a `TORCH_HOME` környezeti változó beállításával a projekt saját `models/` könyvtárába irányít. Ennek segítségével az adott modell csak az első használatakor töltődik le, utána a helyi cache-ből érhető el.

3.8. A felhasznált technológiák

A szoftver Python nyelven készült, amely napjaink egyik legelterjedtebb nyelve gépi tanulási feladatokra, elsősorban a köré épült könyvtárak (PyTorch, TensorFlow, NumPy) ökoszisztémája miatt.

Gépi tanulási keretrendszer

Az FGSM és a PGD algoritmus egyaránt a bemeneti kép szerinti gradiensre épül, amit PyTorch deriválási mechanizmusa számol ki. Emiatt és számos más felhasznált metódus miatt a keretrendszer választása lényegében adott volt. A **torchvision** biztosítja az előtanított ResNet50 és MobileNetV2 modelleket, valamint az ehhez tartozó, ImageNet-1k-n megszokott előfeldolgozó lépéseit (átméretezés, középre vágás, normalizálás). Ez az adathalmaz a képosztályozás egyik legelterjedtebb referencia-adathalmaza, ezért a legtöbb torchvision-modell is erre van előtanítva.

Grafikus felület

A felület a **CustomTkinter** könyvtárra épül, amely a Tkinter, Python beépített GUI keretrendszerének egy modernizált rétege. Azért esett erre a választásom, modernebb funciókat (pl sötét témát, lekerekített gombokat és csúszkákat) ad, miközben a mögöttes eseménykezelési modell megegyezik a natív Tkinterével, így a Python standard könyvtárán kívül nem igényel súlyosabb GUI-függőséget.

Képfeldolgozás

A **Pillow** könyvtár a klasszikus PIL csomag karbantartott továbbfejlesztése. Ez a de facto szabvány képkezelő könyvtár Pythonban, támogat minden elterjedt képformátumot támogat és jól integrálódik a NumPy/PyTorch tenzorokkal. Ez felelős a tenzor és a képfájl közötti átalakításokért, valamint a védekezési modul JPEG-tömörítés és Gauss-elmosás szűrőinek megvalósításáért.

Tesztelési eszközök

A tesztkészlet a **pytest** keretrendszerre épül, amely a Python standard könyvtárban található `unittest`-nél tömörebb, függvényalapú tesztírást tesz lehetővé, illetve fixture-alapú tesztadat-előállítást és parametrizált teszteket is támogat. A diszpécser-függvény viselkedésének ellenőrzéséhez a **pytest-mock** könyvtár `mocker` fixture-je szükséges, amely a valódi implementációt burkoló (*wraps*) helyettesítő objektumok (*spy*) létrehozását teszi egyszerűvé.

Fejlesztői környezet

A függőségek izolálására Python virtuális környezet (`.venv`) szolgál. Ez azért kell, mert a csomagok egymással kompatibilis verziói szükségesek, egy virtuális környezet biztosítja az verzióütközések elkerülését, illetve a működés reprodukálhatóságát. NVIDIA GPU esetén a PyTorch és torchvision CUDA-alapú buildje segítségével opcionálisan tovább gyorsítja mind a klasszifikációt, mind a támadások gradiensszámítását.

3.9. Tervezési döntések és indoklásuk

Adatstruktúrák elválasztása a funkcionális egységektől

A kódban a modellt és a normalizációs paramétereket (`AttackContext`), a betöltött modellt (`ModelBundle`) és egy támadás eredményét (`AttackResult`) tiszta adatstruktúrák (`dataclass`) reprezentálják, a műveletek pedig ezeken dolgoznak, állapot és mellékhatás nélkül. Ez a szétválasztás lehetővé teszi, hogy az egységtesztek a felhasználói felület és a valódi modellek mockolása nélkül fussanak.

Támadás végrehajtása háttérszálon

Az „Attack” gomb megnyomása egy `threading.Thread` példányon indítja el a támadást, mivel egy PGD-támadás már másodperces nagyságrendű futást jelenthet, ami alatt a GUI főszál lefagyna.

FGSM interaktív epsilon-előnézet

Egy sikeres FGSM-támadás után a felület elcache-eli a gradiens előjelét (`attack_engine.fgsm_sign()`), mivel az FGSM képlete ezt az előjelet csak ϵ-nal szorozza meg. Ezzel az ϵ csúszka mozgatásakor csak a már kiszámított előjelet kell újraskálázni, új gradienszámítás nélkül.

Modell-cache és lusta betöltés

A `ModelManager` csak az első ténylegesen kért modellt tölti be, az eredményt egy belső cache-ben tárolja el. Ennek oka, hogy egy modell letöltése és felépítése másodperces nagyságrendű művelet, amit elég egyszer elvégezni.

Kiterjeszthető modellarchitektúra

A `ModelManager.SUPPORTED` lista és a `_build()` metódus egyetlen helyen tartja a rendelkezésre álló modelleket, így egy új torchvision-modell integrálása más kód módosítása nélkül megoldható.

A bemeneti kép 224×224-es mérete

A betöltött kép mérete előbb 256 pixelre kerül átméretezésre, majd 224×224-re középre vágásra (`Resize(256)` + `CenterCrop(224)`), mivel mind a ResNet50, mind a MobileNetV2 erre a méretre lett előtanítva. Erre azért van szükség, mert a nyers képek aránya általában nem négyzetes, egy közvetlen 224×224-re történő átméretezés vizuálisan eltorzítaná a képet. Emiatt viszont a kép szélén lévő tartalom elveszhet. Azt az elvárt felhasználói esetet, hogy a támadott képet visszatöltve ugyanazt az osztályozást kapjuk, az átméretezés és a vágás pontosan 224×224 méretű képre elmarad.

3.10. Az egyes komponensek megvalósítási részletei

A megvalósítási részletek alatt nem célom felfedni az egész program belső működését, csak bemutatok pár érdekesebb részletet és példának az fgsm metódus teljes kódját.

3.10.1. `model_manager.py` modul

- A `ModelBundle` a betöltött hálón (`model`) kívül az ImageNet-1k normalizációs átlagot és szórást is $(1, 3, 1, 1)$ alakú tenzorként tárolja, hogy a támadás során a csatornánkénti normalizálás egyetlen elemenkénti művelettel elvégezhető legyen tetszőleges méretű bemeneti képen.
- A betöltés után a modell minden paraméterén explicit módon `requires_grad_(False)` hívás történik, mivel a támadáshoz kizárólag a bemeneti kép szerinti gradiens ($\nabla_x J$) szükséges, a modell súlyai szerinti gradienst sosem számoljuk ki.
- Ha az eszköz nem explicit adott, akkor a konstruktor `"cuda"`-t választ, amennyiben elérhető CUDA-képes GPU (`torch.cuda.is_available()`), egyébként `"cpu"` lesz az érték.

3.10.2. `attack_engine.py` modul

Először összeállítja a támadást aszerint, hogy melyik algoritmus szükséges és hogy célzott-e a támadás. Ezután hívja meg a tényleges `_fgsm` vagy `_pgd` implementációt, végül az eredményt egységesen egy `AttackResult`-ba csomagolja. A 3.1. kódrészlet az FGSM implementációját mutatja.

```
1  def _fgsm(ctx, x, y, epsilon, targeted):
2      x_adv = x.clone().detach().requires_grad_(True)
3      unnormalized_scores = _forward(ctx, x_adv)
4      loss = F.cross_entropy(unnormalized_scores, y)
5
6      grad = torch.autograd.grad(loss, x_adv)[0]
7      sign = grad.sign()
8      if targeted:
9          x_adv = x_adv - epsilon * sign
10     else:
11         x_adv = x_adv + epsilon * sign
```

```
12
13      x_adv = torch.clamp(x_adv, 0.0, 1.0)
14      return x_adv.detach()
```

3.1. forráskód. Az FGSM implementációja (`attack_engine._fgsm`)

A metódusban mindenek előtt a bemeneti kép tenzorát a `detach()` hívással leválasztjuk a számítási gráfról, majd a `requires_grad_(True)` hívás a tenzort a gráf új levelévé teszi és bekapcsolja rá a gradienskövetést, hogy a következő forward lépés után a bemeneti kép szerint is deriválható legyen. A `torch.autograd.grad` hívás a veszteség bemeneti kép szerinti gradiensét adja vissza egyetlen hátrafelé irányuló lépésben, ez a korábban használt jelölés szerinti $\nabla_x J(\theta, x, y)$-t. Ezután a célzott és normál támadás közti elágazás következik, ahol a kifejezés csak egy előjelben különbözik. A visszatérési értéket a `clamp` alkalmazásával szorítjuk vissza az $[0, 1]$ pixeltartományra, majd a végén egy `detach()` hívás biztosítja, hogy a visszaadott tenzor ne hordozzon további autograd-gráf hivatkozást.

3.10.3. `defense_module.py` modul

A védekezés PIL-alapú műveletekből áll, ezért a modul minden hívás előtt és után elvégez egy, a pixeltér-tenzor és a PIL kép közötti konverziót.

- A JPEG-tömörítés egy memóriában tartott bájtpuffert (`io.BytesIO`) használ, ebbe kódolja be a képet a megadott minőséggel, majd ugyaninnen dekódolja azt vissza. Tehát nem történik lemezre írás, így ténylegesen egy veszteséges tömörítés-visszafejtés történik, nem csak annak szimulációja.
- A Gauss-elmosás a PIL `ImageFilter.GaussianBlur` szűrőjét alkalmazza a megadott sugárral.

3.10.4. `utils.py` modul

A nyers δ perturbáció felhasználói felületen ábrázolásához pár részlet. Mivel értékei jellemzően nagyon kis abszolút értékűek, ennek megjelenítése gyakorlatilag egyszínű, szemmel kivehetetlen képet adna. A `perturbation_to_pil` függvény ezért a perturbációt egy `magnify` szorzóval felnagyítja, majd 0,5-tel eltolja, mielőtt a $[0, 1]$ tartományra vágná. Az eltolás azért szükséges, hogy ábrázoljuk a negatív előjelű változást, különben azok a $[0, 1]$ tartományra vágásnál elvesznének. Az eltolás miatt egy nulla közeli (gyakorlatilag

változatlan) pixel középszürkeként jelenik meg, míg a pozitív és negatív irányú változások világosabb, illetve sötétebb árnyalatként válnak láthatóvá.

4. fejezet

Tesztelés

4.1. Egységtesztek

A program tesztkészletének felépítésénél a szakirodalomból ismert *tesztpiramisnak* nevezett hierarchiát követtem [10]. Ennek alapját a gyorsabb és kevés komponenst érintő tesztek, az egységtesztek képezik, melyek a lehető legkisebb elkülöníthető egységek tesztelését valósítják meg. Futási idejük jellemzően ezredmásodperces tartományban mozog és akár lefedhetik az input paraméterek teljes értékkészletét. Az elkészítésüknél fontos szempont a többi egységtől való függetlenség (izoláció) és az perifériáktól való függetlenség, I/O, adatbázis, hálózat, stb. Ezeknek a teszteknek az elkészítésére érdemes nagy figyelmet fordítani, mivel a korai szakaszban feltárt hibák javítása általában lényegesen olcsóbb, mint a később, integráció vagy éles környezetben felfedezetteké.

Az egységtesztek tudatosan nem a valódi, előtanított ResNet50/MobileNetV2 modelleket használják. Az `attack_engine` teszteknél egy `tiny_model` fixture ad egy 5 osztályos, 8×8 pixeles bemenetet feldolgozó, egyetlen lineáris rétegből álló hálót. Ez a döntés a tesztek sebességét (a valódi háló letöltése és felépítése másodperces nagyságrendű lenne) és izoláltságát szolgálja: a teszt kizárólag a saját kódunk logikáját vizsgálja, nem a harmadik féltől származó architektúra pontosságát.

Minden véletlent felhasználó fixture (pl. a bemeneti képet generáló `sample_image_tensor`) maga hívja meg a `torch.manual_seed`-et közvetlenül az adat előállítása előtt, így a determinizmus független a fixture-ök végrehajtási sorrendjétől. Az egységtesztek szintjén nem támadási sikerrátát mérünk (ehhez sok mintás, statisztikai kiértékelés kellene), hanem az algoritmusok tulajdonságait ellenőrizzük konkrét, kis méretű bemeneteken.

Ebben a részben a számosságuk miatt nem mutatom be az összes egységtesztet, csak a középpontban álló `attack_engine.py` modult mivel ez valósítja meg az alkalmazás lényegi logikáját (az FGSM és PGD algoritmusokat, valamint az ezeket kiszolgáló `predict()` és `attack()` függvényeket).

A `predict()` és a diszpécser (`attack()`) tesztjei

`test_predict_returns_topk_shapes`: Ellenőrzi, hogy `topk=3` esetén a visszaadott valószínűség- és index-tenzor alakja $(1, 3)$, azaz a batch- és k-dimenzió megfelel a kérésnek.

`test_predict_returns_valid_distribution`: A `topk=5` valószínűségek mindegyike a $[0, 1]$ intervallumba esik, és – mivel az 5 osztályos `tiny_model` minden osztálya lekérdezésre kerül – összegük 1,0-ra `allclose`. Ez a softmax kimenet valódi valószínűségi eloszlás voltát igazolja.

`test_predict_topk_returns_highest_first`: A visszaadott valószínűségek csökkenő sorrendben szerepelnek ($\text{probs}[0, i] \geq \text{probs}[0, i+1]$), vagyis a `topk` valóban a legnagyobb konfidenciájú osztályokat adja vissza, méghozzá rendezetten.

`test_attack_unknown_method_raises`: Határeset: egy nem létező módszernév (`"NEM VALID"`) megadásakor `ValueError`-t vár, mielőtt bármilyen gradiensszámítás történne – a bemenet-validáció a drága számítás előtt fut le.

`test_attack_fgsm_ignores_iterations_argument`: Annak ellenére, hogy a hívó `iterations=99`-et ad át, FGSM esetén a visszaadott `AttackResult.iterations_run` értéke 1 – igazolva, hogy az FGSM valóban egylépéses, és a paramétert a diszpécser figyelmen kívül hagyja erre a módszerre.

`test_attack_pgd_reports_requested_iteration_count`: PGD esetén a kért iterációszám (4) változatlanul megjelenik az `iterations_run` mezőben, alátámasztva, hogy a diszpécser a paramétert továbbadja a `_pgd` implementációnak.

`test_attack_fgsm_dispatches_only_to_fgsm`: `mocker.patch(..., wraps=_fgsm)` és `wraps=_pgd` spy-okkal figyeli a hívásokat: `method="FGSM"` esetén `_fgsm.assert_called_once()` és `_pgd.assert_not_called()` teljesül. A `wraps` paraméter miatt a valódi implementáció is lefut, tehát az `AttackResult` összeállítása is ellenőrzött marad.

`test_attack_pgd_dispatches_only_to_pgd`: Az előző teszt tükörképe: `method="PGD"` esetén kizárólag `_pgd` hívódik meg, `_fgsm` nem. A két teszt együtt zárja ki, hogy a diszpécser valamelyik ágon „átcsorogva” mindkét algoritmust lefuttassa.

Az FGSM implementáció (`_fgsm`) tesztjei

`test_fgsm_targeted_steps_opposite_direction_of_untargeted`: Azonos bemeneten és ϵ mellett a célzott és nem célzott perturbáció pontosan ellentétes előjelű ($x_{\text{nem célzott}} - x = -(x_{\text{célzott}} - x)$), ami közvetlenül a matematikai definícióból (a gradiens előjelének megfordításából) következik.

`test_fgsm_clamps_to_valid_pixel_range`: Szélsőséges bemenet (csupa nulla pixel) és nagy $\epsilon = 0{,}5$ mellett is a kimenet minden eleme a $[0, 1]$ tartományban marad – igazolva, hogy a `clamp` lépés ténylegesen érvényesül, nem csak elméletben szerepel a specifikációban.

`test_fgsm_result_is_detached_from_autograd`: A visszaadott tenzor `requires_grad` attribútuma `False`.

`test_fgsm_result_has_no_graph_history`: Szigorúbb bizonyíték az előzőnél: `grad_fn is None` *és* `is_leaf is True` egyszerre teljesül, ami kizárja azt az esetet is, amikor egy tenzor `requires_grad` értéke hamis, de mégis egy számítási gráf levele marad rejtett hivatkozásokkal.

`test_fgsm_zero_epsilon_returns_original`: Határeset: $\epsilon = 0$ esetén a perturbáció nulla, a kimenet `allclose` az eredeti képpel – az algoritmus degenerált esetben identitásként viselkedik.

`test_fgsm_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték: a `predict()`-en keresztül lekérdezett, a valódi címkéhez tartozó konfidencia a támadás *után* szigorúan kisebb, mint előtte – ez nem csak a perturbáció létezését, hanem annak tényleges hatását igazolja a modell kimenetére.

`test_fgsm_targeted_increases_confidence_in_target_label`: A célzott változat tükörpárja: a mesterségesen kiválasztott célosztály konfidenciája nő a támadás hatására, alátámasztva, hogy a célzott mód valóban a célosztály felé tereli a predikciót, nem csupán „valamerre" téveszti meg a modellt.

A PGD implementáció (`_pgd`) tesztjei

`test_pgd_result_stays_within_epsilon_ball`: Mérsékelt lépésméret ($\alpha = 0{,}02$) mellett a végső perturbáció minden pixelre $|\delta| \leq \epsilon + 10^{-6}$ (a tolerancia a lebegőpontos kerekítést kompenzálja) – ez az ℓ_∞-gömbbe való projekció alapvető helyességét igazolja.

`test_pgd_hits_ball_when_step_too_large`: Szándékosan irreálisan nagy lépésméretet ($\alpha = 10{,}0 \gg \epsilon = 0{,}02$) ad meg, ami minden iterációban túllövést okoz; ilyenkor a projekciós `clamp`-nek kell egyedül biztosítania a korlátot, ezért a tesztben a végső $|\delta|$ pontosan (nem csak $\leq$) ϵ-nak adódik minden pixelen – ez a projekció, nem a lépésméret helyes megválasztása általi korlátozottságot bizonyítja.

`test_pgd_clamps_to_valid_pixel_range`: A pixeltartomány-korlátozás ugyanúgy teljesül PGD esetén is, mint FGSM-nél: szélsőséges (nulla) bemeneten és nagy ϵ mellett a kimenet a $[0, 1]$ tartományban marad.

`test_pgd_result_has_no_graph_history`: Ugyanaz az erős `detach`-bizonyíték (`grad_fn is None` és `is_leaf is True`), mint FGSM esetén – fontos, mert a PGD több iterációt is végez, minden lépésben új `requires_grad_(True)` hívással, így könnyebben „elszivároghatna" egy hivatkozás a gráfra, ha a ciklus végén elmaradna a leválasztás.

`test_pgd_zero_epsilon_returns_original`: Határeset: $\epsilon = 0$ esetén – annak ellenére, hogy több iteráció is lefut – a projekció minden lépésben nulla szélességű intervallumra szorítja a perturbációt, így a végeredmény megegyezik az eredeti képpel.

`test_pgd_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték `predict()`-en keresztül: a valódi címke konfidenciája több iteráció után is csökken – ugyanaz az elv, mint FGSM-nél, de itt azt is igazolja, hogy az iteratív frissítés nem „rontja el" a kumulatív hatást.

`test_pgd_targeted_increases_confidence_in_target_label`: A célzott PGD a célosztály konfidenciáját növeli – ugyanazon tulajdonságpár (nem célzott csökkenti/célzott növeli a releváns konfidenciát) FGSM-re és PGD-re is parametrizált formában lett tesztelve (`@pytest.mark.parametrize`), elkerülve, hogy csak az egyik algoritmusra íródjon le a viselkedés.

4.2. Integrációs tesztek

Az integrációs tesztek mockolás nélkül, több valódi modul együttműködését ellenőrzik. A `tests/integration/test_pipeline.py` fájlban három tesztet készítettem. Az első kettő a 224×224 pixeles `tiny_model_224` hálót használja, ezért gyors, és nem igényel letöltött súlyokat. A harmadik a valódi, előtanított MobileNetV2 modellt tölti be, és kihagyásra kerül (`skipif`), ha a modell súlyfájlja nincs a helyi gyorsítótárban. Így a tesztkészlet futtatása sosem indít hálózati letöltést.

Védekezés a támadott képen

`test_defense_shrinks_perturbation_of_attacked_image`: Egy képfájl betöltését, a PGD támadást, a Gauss-elmosást és a `predict()` hívást fűzi össze. A betöltött kép egyszínű, így a támadás előjel-alapú perturbációja tisztán nagyfrekvenciás zaj. Az elmosás után a kép távolsága az eredetitől (az L_2 normában) kisebb, mint a támadott képé volt. Emellett a védett kép alakja és pixeltartománya érvényes marad, a rá számított valószínűségek összege pedig 1. Ez a teszt a dolgozat központi állítását, a védekezés hatását a támadásra, a modulok közötti valódi adatáramláson igazolja.

`test_saved_adversarial_image_survives_disk_roundtrip`: Az FGSM támadás eredményét a felület mentési folyamatának megfelelően PNG fájlba írja (a `build_adversarial_filename` által generált névvel), majd a `load_image_as_tensor` függvénnyel újra betölti. Az újratöltött kép és a memóriában lévő támadott kép között csak a 8 bites kvantálásból adódó eltérés engedett meg. A perturbáció a betöltés és mentés után is az ϵ-sugarú környezeten belül marad. Ez igazolja, hogy a letöltött támadott kép később ugyanolyan pixelpontossággal visszatölthető a programba.

`test_real_mobilenet_pgd_changes_prediction_within_budget`: A `ModelManager` által betöltött valódi MobileNetV2 modellből a `ModelBundle` adataival felépíti az `AttackContext`-et, ahogyan a grafikus felület is teszi. Ezután 10 lépéses PGD támadást futtat. Ellenőrzi, hogy a perturbáció az $\epsilon = 8/255$ korláton belül marad, hogy a modell legvalószínűbb osztálya megváltozik, hogy a kimenet 1000 osztályos, és hogy az eredeti osztály valószínűsége csökken. Ez az egyetlen teszt, amely a valódi modell és a támadómotor illesztését (normalizálás, eszköz, osztályszám) próbálja ki, ezért az egységtesztekben mockolt `ModelManager` hiányosságát pótolja.

A valódi modellen szándékosan nem ellenőrzöm, hogy a védekezés visszaállítja-e az eredeti osztályt. Kézzel előállított, szintetikus képen ez nem tulajdonság, hanem statisztikai eredmény, amelyet egy kis mintán nem lehet megbízhatóan tesztelni.

4.3. Automatizált GUI tesztek

A GUI tesztek a felhasználói felületet magát, a ténylegesen megjelenő ablakot és widgeteket vizsgálják, programozott módon szimulálva a felhasználói interakciókat, majd ellenőrizi, hogy a felület a várt módon reagál-e. Ez a legfelső, egyben legköltségesebb és leglassabb réteg a tesztpiramisban, hiszen minden egyes esethez fel kell építeni a teljes grafikus felületet – ezért csak kevés, a legfontosabb felhasználói forgatókönyveket lefedő tesztesetre érdemes szorítkozni, a részletesebb logikai ellenőrzést az alsóbb rétegekre (egység- és integrációs tesztek) hagyva.

4.4. Funkcionális tesztek és felhasználói validáció

A grafikus felület funkcionális, „fekete doboz” teszteseteit a felhasználói forgatókönyvek alapján, *Given–When–Then* formában fogalmazom meg. Az alábbi esetek a tényleges felhasználói felületének elemeihez (gombok, csúszkák, célosztály-kereső) igazodnak, és kézi (manuális) végrehajtásra tervezettek.

- **Kép betöltése a „Load Image” gombbal**
  AS A: felhasználó I WANT TO: egy képet betölteni a vizsgálathoz
  GIVEN: az alkalmazás elindítva, még nincs kép betöltve
  WHEN: a „Load Image” gombra kattint, majd egy érvényes képfájlt választ ki
  THEN: az „Original” képmezőben megjelenik a beolvasott, 224×224-re vágott kép, és alatta a top-3 predikció és konfidencia frissül

- **Modell váltása betöltött kép mellett**
  AS A: felhasználó I WANT TO: összehasonlítani, hogy más architektúra hogyan látja a képet
  GIVEN: egy kép már be van töltve, az „Original” predikciós panel ki van töltve
  WHEN: a modell-választó legördülőben „MobileNetV2”-t választ „ResNet50” helyett
  THEN: az „Original” predikciók újraszámolódnak az új modellel; ha korábban már futott támadás, az „Adversarial” predikciók is frissülnek

- **Nem célzott támadás végrehajtása**
  AS A: felhasználó I WANT TO: általános tévesztést előidézni a modellben
  GIVEN: kép betöltve, „Targeted attack” jelölőnégyzet kikapcsolva, ϵ és iterációszám csúszkák beállítva
  WHEN: az „Attack” gombra kattint
  THEN: a státuszsor „Running...” majd „Attack complete” üzenetet mutat, megjelenik a felnagyított perturbáció és a támadott kép, és az „Adversarial” predikciós panelen az eredeti top-1 osztály konfidenciája jellemzően csökken

- **Célzott támadás célosztály kereséssel**
  AS A: felhasználó I WANT TO: egy konkrét osztályra rávenni a modellt
  GIVEN: kép betöltve, a célosztály-kereső mezőbe elkezd gépelni egy osztálynevet
  WHEN: a szűrt listából kiválaszt egy osztályt, majd az „Attack” gombra kattint
  THEN: a „Targeted attack” jelölőnégyzet automatikusan bejelölődik, a státuszsor a kiválasztott osztály nevét mutatja, és a támadás után a célosztály konfidenciája nő

- **Epsilon csúszka élő előnézete FGSM módban**

  AS A: felhasználó I WANT TO: interaktívan látni, hogyan változik a perturbáció erőssége

  GIVEN: FGSM módszerrel már lefutott egy támadás (a gradiens előjele cache-elve van)

  WHEN: az ϵ csúszkát új értékre húzza, PGD-re váltás nélkül

  THEN: a perturbáció és az ellenfél alapú kép új gradiens-számítás *nélkül*, azonnal újraszámolódik a cache-elt előjel alapján

- **Attack gomb kép nélkül**

  AS A: felhasználó I WANT TO: (véletlenül) támadást indítani kép betöltése előtt

  GIVEN: az alkalmazás elindítva, nincs betöltött kép

  WHEN: az „Attack" gombra kattint

  THEN: egy „No image" információs párbeszédablak jelenik meg, a támadás nem indul el

- **Védekezési szűrő alkalmazása (Blur / JPEG)**

  AS A: felhasználó I WANT TO: megvizsgálni, hogy egy egyszerű szűrő semlegesíti-e a zajt

  GIVEN: egy sikeres támadás után az „Adversarial" kép elérhető

  WHEN: a „Defend (Blur)" vagy „Defend (JPEG)" gombra kattint

  THEN: az „Adversarial" képmező a szűrt képre frissül, és a predikciós panel az újraosztályozott (a szűrés után kapott) eredményeket mutatja

- **Defend gomb ellenfél alapú kép nélkül**

  AS A: felhasználó I WANT TO: (véletlenül) védekezést alkalmazni támadás előtt

  GIVEN: kép be van töltve, de még nem futott támadás

  WHEN: valamelyik „Defend" gombra kattint

  THEN: egy „No adversarial" információs párbeszédablak jelenik meg, semmilyen kép nem változik

- **Az eredeti kép letöltése**

  AS A: felhasználó I WANT TO: az eredeti képet fájlba menteni

  GIVEN: egy kép be van töltve

  WHEN: a „Download original" gombra kattint

  THEN: mekapja a mentett 224×224 pixeles fájlt

- **A támadott kép letöltése és visszatöltése**

  AS A: felhasználó I WANT TO: az ellenfél alapú képet elmenteni és újra feltölteni

  GIVEN: egy sikeres támadás után az „Adversarial” kép létrejön

  WHEN: a „Download adversarial” gombra kattint, elmenti PNG-ben, majd a „Load Image” gombbal visszatölti

  THEN: a visszatöltött kép ugyanazt a top-1 osztályozást adja, a konfidencia pedig csak minimálisan tér el

- **Letöltés kép vagy támadás nélkül**

  AS A: felhasználó I WANT TO: (véletlenül) letöltést kérni, mielőtt volna mit menteni

  GIVEN: nincs betöltött kép, illetve van kép, de még nem futott támadás

  WHEN: a „Download original” vagy a „Download adversarial” gombra kattint

  THEN: egy „No image” vagy „No adversarial” információs párbeszédablak jelenik meg, fájl nem készül

- **Reset gomb**

  AS A: felhasználó I WANT TO: új kísérletet kezdeni ugyanazzal a képpel

  GIVEN: egy támadás és esetleg egy védekezés már lefutott

  WHEN: a „Reset” gombra kattint

  THEN: a perturbáció és az „Adversarial” képmező üresre vált, az „Adversarial” predikciós panel névtelen („—”) állapotba áll vissza, az „Original” kép és predikciói változatlanok maradnak

- **Hibakezelés támadás közben**

  AS A: felhasználó I WANT TO: érthető visszajelzést kapni, ha a támadás valamiért meghiúsul

  GIVEN: kép betöltve, a háttérszálon futó `_attack_worker` kivételt dob (pl. eszközhiba miatt)

  WHEN: az „Attack” gombra kattint

  THEN: egy „Attack failed” hibaablak jelenik meg a teljes veremkiíratással, a státuszsor „Attack failed.” szöveget mutat, az alkalmazás nem fagy le

4.5. Teszteredmények és értelmezésük

4.6. A tesztelés korlátai

5. fejezet

Összefoglalás

5.1. Elért eredmények

5.2. Következtetések

5.3. Továbbfejlesztési lehetőségek

Irodalomjegyzék

[1] Ian J. Goodfellow, Jonathon Shlens és Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572.

[2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras és Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2018. arXiv: 1706.06083.

[3] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow és Rob Fergus. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199.

[4] Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies és Maia Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST AI 100-2e2025. 2025. URL: https://doi.org/10.6028/NIST.AI.100-2e2025.

[5] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi és Pascal Frossard. *Universal adversarial perturbations*. 2017. arXiv: 1610.08401.

[6] Mahmood Sharif, Sruti Bhagavatula, Lujo Bauer és Michael K. Reiter. *Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition*. Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS 2016). 2016. DOI: 10.1145/2976749.2978392.

[7] Kevin Eykholt, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno és Dawn Song. *Robust Physical-World Attacks on Deep Learning Visual Classification*. 2018. arXiv: 1707.08945.

[8] Shang-Tse Chen, Cory Cornelius, Jason Martin és Duen Horng Chau. *ShapeShifter: Robust Physical Adversarial Attack on Faster R-CNN Object Detector*. 2018. arXiv: 1804.05810.

[9] Gintare Karolina Dziugaite, Zoubin Ghahramani és Daniel M. Roy. *A study of the effect of JPG compression on adversarial images*. 2016. arXiv: 1608.00853.

[10] Martin Fowler. *TestPyramid*. martinfowler.com. 2012. URL: https://martinfowler.com/bliki/TestPyramid.html.